

ISRO Route Resilience - Complete Project Report

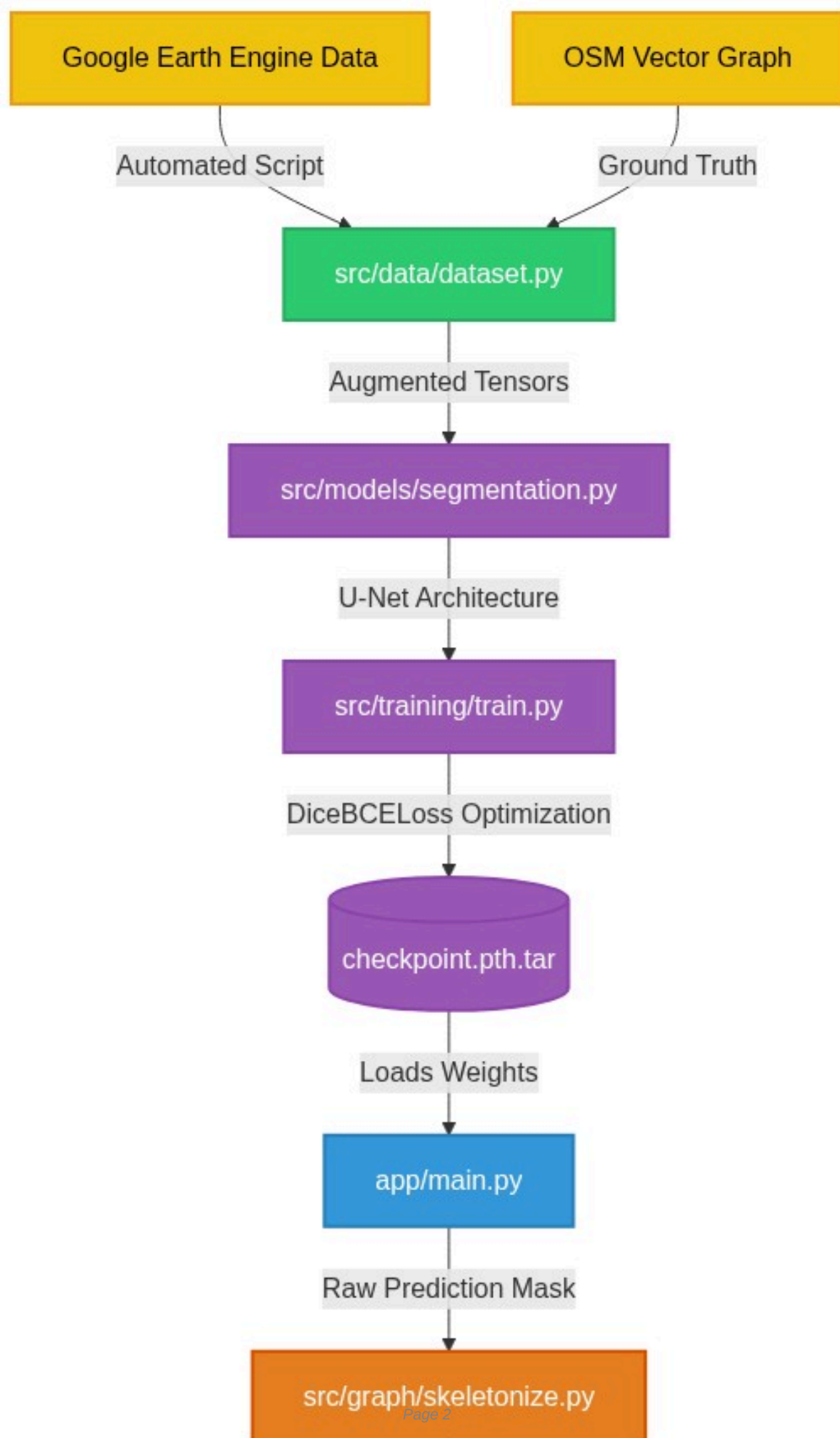
1. Project Overview & Objective

This project is engineered for the **ISRO NNRMS Hackathon**. The primary goal is to extract road networks from noisy satellite imagery (where roads are blocked by trees or clouds) and mathematically analyze the network's resilience to disasters.

Standard AI fails when a tree covers a road (it predicts a broken road). Our project uses a hybrid architecture: a **Deep Learning model** (U-Net) to find the roads, and a **Graph Theory Topological Engine** (Minimum Spanning Tree) to automatically "heal" the broken roads and calculate the impact if a critical node fails (e.g., flooding).

2. Complete Pipeline Diagram

ISRO Route Resilience - Complete Project Report



ISRO Route Resilience - Complete Project Report

3. Technologies Used & Their Purpose

- **PyTorch & U-Net (Machine Learning):** Used to train the neural network to perform pixel-level semantic segmentation (identifying exactly which pixels are roads).
- **NetworkX & Scipy (Graph Theory):** Used to convert the AI's pixel output into a mathematical graph (nodes and edges) to run topology algorithms like Betweenness Centrality.
- **Streamlit & Folium (Frontend):** Used to create the beautiful glassmorphism web dashboard and interactive maps.
- **Albumentations (Data Augmentation):** Used to inject artificial noise (shadows, rotations) into the training data so the AI learns to see through bad weather.
- **Google Earth Engine / geemap (Data Pipeline):** Used to automatically pull cloud-free Sentinel-2 satellite imagery.

4. Detailed File-By-File Breakdown

Frontend & User Interface

app/main.py

What it is: The core User Interface file powered by Streamlit.

Why we used it: It acts as the bridge between the user and the complex math models. It contains custom CSS to make the app look stunning (glassmorphism cards). It loads the AI weights, runs inference on a user's uploaded satellite image, and renders the healed graph interactively on a Leaflet map. Crucially, it handles the "Disaster Simulation" logic where a user disables a node to see the Travel Time Delay.

Machine Learning (Model & Training)

src/models/segmentation.py

What it is: Defines the exact mathematical architecture of the AI.

Why we used it: It implements a standard U-Net with skip connections. This is the industry standard for biomedical and satellite segmentation because it preserves high-resolution details (essential for thin roads).

src/training/train.py & src/training/losses.py

What it is: The engine that teaches the AI how to find roads.

ISRO Route Resilience - Complete Project Report

Why we used it: Because roads make up only 5% of a satellite image, standard accuracy metrics fail. The **losses.py** file implements **DiceBCELoss** to force the AI to care about thin roads. **train.py** uses the Adam optimizer and PyTorch Automatic Mixed Precision to drastically speed up training on Kaggle GPUs. It outputs the **checkpoint.pth.tar** weights.

Data Processing Pipeline

data_pipeline.py & src/data/dataset.py

What it is: The scripts that feed information to the AI.

Why we used it: **data_pipeline.py** connects to Google Earth Engine to automatically pull raw Sentinel-2 imagery and matches it with OpenStreetMap vectors for zero-manual-effort dataset creation. **dataset.py** is the PyTorch DataLoader that applies Albumentations (fake shadows, dropouts) to mathematically torture the AI so it becomes robust against real-world noise.

Graph Theory & Topology (The Healing Engine)

src/graph/skeletonize.py

What it is: Thins the thick AI prediction down to a 1-pixel wide line using the Medial Axis Transform.

Why we used it: You cannot run graph math on thick blobs of pixels. It must be perfectly thin.

src/graph/topology.py

What it is: The core "Healing" algorithm.

Why we used it: If a tree covers a road, the AI draws a gap. This script converts the pixels to a NetworkX graph, uses a Disjoint Set (Union-Find) and **Kruskal's Minimum Spanning Tree (MST)** to mathematically hunt down broken dead-ends and stitch them together using the shortest possible path, completely healing the occlusion.

src/graph/analysis.py & src/graph/ablation.py

What it is: The disaster stress-testing modules.

Why we used it: **analysis.py** calculates "Betweenness Centrality" to find bottlenecks (major bridges). **ablation.py** simulates a catastrophic failure by deleting a bottleneck node and calculates the "Resilience Index" (ratio of average shortest paths before vs. after failure) to tell ISRO how vulnerable the network is.